

Fertilizer Recommendation System For Agriculture Using Ai

1.Introduction

1.1Project Overview

The **Fertilizer Recommendation System for Agriculture using AI** aims to provide farmers with optimized fertilizer recommendations to enhance crop yield, maintain soil health, and promote sustainable farming practices. This system analyzes various parameters, including soil composition, pH levels, moisture content, weather conditions, and crop types, to suggest the right type and quantity of fertilizers. By leveraging machine learning models trained on historical agricultural data, it delivers personalized, data-driven recommendations that minimize over-fertilization and reduce environmental impact. The system can be accessed through a user-friendly mobile or web application, where farmers input details about their crops and fields to receive real-time suggestions. Additionally, it integrates weather data for context-aware guidance and offers a feedback mechanism for continuous improvement. This project not only simplifies decision-making for farmers but also supports cost-effective and eco-friendly agricultural practices, contributing to improved productivity and long-term soil sustainability.

1.2 Project Objective

The objective of the **Fertilizer Recommendation System for Agriculture using AI** is to create an intelligent and data-driven system that assists farmers in optimizing fertilizer usage for improved crop yield and soil health. By analyzing crucial parameters such as soil pH, nutrient levels, moisture content, crop type, and weather conditions, the system provides personalized recommendations for the type and quantity of fertilizers required. This helps farmers make informed decisions that boost productivity while avoiding the negative effects of over-fertilization, such as soil degradation and environmental pollution.

Additionally, the project aims to promote cost-efficient and sustainable farming practices. By reducing unnecessary fertilizer expenses and encouraging balanced nutrient management, the system supports eco-friendly agricultural methods. Through a user-friendly interface, such as a mobile or web application, farmers can input field-specific data and receive real-time suggestions tailored to their needs. This initiative not only simplifies decision-making for farmers but also contributes to long-term agricultural sustainability and global food security.

2. Project Initialization and Planning Phase

The initialization and planning phase of the “Fertilizer Recommendation System for Agriculture using AI” involves defining the project scope and objectives, conducting research on fertilizers, soil health, and crop requirements, and gathering data from sources like plant_images datasets and weather APIs. Stakeholder inputs from farmers and agricultural experts are incorporated to align the system with practical needs. A comprehensive plan is developed, detailing tasks, timelines, and milestones, along with the selection of machine learning models and required technologies. Risks are assessed, resources allocated, and a prototype is planned to test feasibility, ensuring a structured approach for successful implementation.

Activity 1: Define Problem Statement

The first step in the project is to define the problem statement, which focuses on the challenges farmers face in managing fertilizers, such as inaccurate decisions regarding type, quantity, and timing of application. These issues lead to reduced crop yields, degraded soil health, higher costs, and environmental harm. The problem statement highlights the need for an AI-driven system that provides personalized fertilizer recommendations based on soil properties, crop requirements, and environmental factors, aiming to optimize fertilizer use, enhance productivity, lower costs, and promote sustainable farming practices.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Problem Statement Report:
[ClickHere](#)

Activity 2: Project Proposal (Proposed Solution)

The proposed solution is to develop an AI-driven Fertilizer Recommendation System that provides personalized fertilizer suggestions based on soil properties, crop type, and environmental factors. By leveraging machine learning models trained on historical data, soil nutrient information, and weather conditions, the system will offer accurate recommendations regarding the type and quantity of fertilizers needed for optimal crop growth. Farmers will input field-specific data through a user-friendly mobile or web app, and the system will generate tailored advice to minimize over-fertilization, reduce costs, and promote sustainable farming practices. The solution will also include a feedback mechanism for continuous improvement, supporting eco-friendly and productive agriculture.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Project Proposal Report:
[ClickHere](#)

Activity 3: Initial Project Planning

The initial project planning involves outlining the project scope, objectives, and key deliverables while defining tasks, timelines, and milestones. This activity includes selecting suitable technologies, such as machine learning frameworks and data processing tools, and identifying data sources like soil datasets, weather APIs, and agricultural guidelines. Resource allocation, including team roles and responsibilities, is determined, and potential risks, such as data integration challenges, are assessed. Additionally, a prototype plan is developed to test the system's feasibility, ensuring a structured approach for the successful execution of the project.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Initial Project Planning Report: [ClickHere](#)

3. Data Collection and Preprocessing Phase

The Data Collection and Preprocessing Phase involves executing a plan to gather relevant data from Kaggle, ensuring data quality through verification and addressing missing values. Preprocessing tasks include cleaning, encoding, and organizing the dataset for subsequent exploratory analysis and machine learning model development.

Activity 1: Data Collection Plan, Raw Data Sources Identified

The data collection plan focuses on identifying and gathering relevant raw data from reliable sources to train and develop the **Fertilizer Recommendation System**. Key data sources include soil datasets containing information on pH, nutrient levels, and moisture content, weather APIs for real-time data on rainfall, temperature, and humidity, and agricultural datasets detailing crop-specific nutrient requirements. Additional data may be collected from government databases, research publications, and field surveys to ensure comprehensive coverage. This activity ensures that the system has access to high-quality data for accurate analysis and recommendations.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Raw Data Sources Report:
[ClickHere](#)

Activity 2: Data Quality Report

The data quality report evaluates the collected data to ensure its reliability, completeness, and suitability for developing the **Fertilizer Recommendation System**. It involves checking for missing values, inconsistencies, duplicate entries, and outliers in soil datasets, weather data, and crop-specific nutrient information. The report assesses data accuracy, relevance, and timeliness, ensuring it aligns with the project's requirements. Identified issues are documented, and strategies like data cleaning, normalization, and enrichment are proposed to improve data quality, guaranteeing robust and precise recommendations in the final system.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Data Quality Report:
[ClickHere](#)

Activity 3: Data Exploration and Preprocessing

The data exploration and preprocessing phase involves analyzing the collected data to understand its structure, patterns, and relationships while preparing it for use in the **Fertilizer Recommendation System**. Key activities include visualizing data distributions, identifying correlations between variables (e.g., soil nutrients and crop yields), and detecting anomalies or missing values. Preprocessing steps include cleaning the data, normalizing values, encoding categorical variables, and splitting the dataset into training and testing sets. This phase ensures the data is consistent, relevant, and ready for effective machine learning model development.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Data Exploration and Preprocessing Report: [ClickHere](#)

4. Model Development Phase

The model development phase involves building and training machine learning models for the **Fertilizer Recommendation System** to predict the type and quantity of fertilizers required based on inputs such as soil properties, crop type, and weather conditions. Suitable algorithms, such as regression, decision trees, or deep learning models, are selected and trained on preprocessed data. Hyperparameter tuning is conducted to optimize performance, while evaluation metrics like accuracy and RMSE are used to assess the models. The best-performing model is validated using test data to ensure reliability and scalability for accurate recommendations.

Activity 1: Initial Model Training Code, Model Validation and Evaluation Report

The initial model training involves implementing machine learning code to train the selected model on the preprocessed dataset, using algorithms such as regression, decision trees, or deep learning. The model is trained to predict fertilizer recommendations based on soil, crop, and environmental inputs. Once trained, the model undergoes validation using a separate test dataset to evaluate its performance through metrics like accuracy, precision, recall, and RMSE. An evaluation report is prepared, summarizing the model's strengths, weaknesses, and areas for improvement, ensuring a foundation for refining the model in subsequent iterations.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Model Development Phase Template: [ClickHere](#)

Activity 2: Model Selection Report

The model selection report documents the process of evaluating and selecting the most suitable machine learning model for the **Fertilizer Recommendation System**. Multiple algorithms, such as regression models, decision trees, random forests, and deep learning models, are tested based on their performance metrics, including accuracy, RMSE, and precision. The report compares these models, highlights their strengths and weaknesses, and justifies the final selection based on factors like prediction accuracy, scalability, and computational efficiency. This ensures the chosen model aligns with the system's requirements and delivers reliable and precise recommendations.

Ref. template: [Click Here](#)

Fertilizer Recommendation System for Agriculture using AI Model Selection Report: [ClickHere](#)

5. Model Optimization and Tuning Phase

The Model Optimization and Tuning Phase involves refining machine learning models for peak performance. It includes optimized model code, fine-tuning hyperparameters, comparing performance metrics, and justifying the final model selection for enhanced predictive accuracy and efficiency.

Activity 1: Hyperparameter Tuning Documentation

The **Fertilizer Recommendation System for Agriculture using AI** underwent a comprehensive hyperparameter tuning process to enhance the model's performance. Key hyperparameters, including learning rate, batch size, and the number of epochs, were optimized to ensure accurate predictions of fertilizer requirements based on soil, crop, and environmental factors. The final model, after fine-tuning, demonstrated high accuracy, low prediction error, and strong generalization capabilities, making it suitable for real-time agricultural decision-making. This optimized system is designed to provide precise and reliable fertilizer recommendations, ultimately supporting sustainable farming practices and improving crop yields.

Ref. template: [Click Here](#)

Blueberry yield Hyperparameter Tuning Report: [ClickHere](#)

6.RESULT

Prediction Result

Precaution for the Disease:

Remove affected leaves, apply fungicides like azoxystrobin, and avoid splashing water on leaves.

[Back to Home](#)[Predict Another](#)

Prediction Result

Precaution for the Disease:

Use fungicides like chlorothalonil, remove infected leaves, and practice crop rotation.

[Back to Home](#)[Predict Another](#)

Prediction Result

Precaution for the Disease:

Maintain proper plant nutrition, use resistant varieties, and ensure good airflow around plants.

[Back to Home](#)[Predict Another](#)

7.ADVANTAGES AND DISADVANTAGES

ADVANTAGES:

1. **Improved Crop Yield:** By providing optimized fertilizer recommendations, the system helps farmers increase crop productivity through better nutrient management.
2. **Cost Efficiency:** Farmers can reduce fertilizer expenses by applying the right amount of fertilizer, avoiding overuse and minimizing waste.
3. **Sustainability:** The system promotes eco-friendly practices by minimizing environmental impact, such as reducing soil degradation and preventing water contamination from excess fertilizer use.
4. **Personalized Recommendations:** It offers tailored fertilizer plans based on soil health, crop type, and weather conditions, ensuring more accurate and efficient results.
5. **Ease of Use:** The system's user-friendly mobile or web interface makes it accessible for farmers with varying levels of technical expertise.
6. **Data-Driven Insights:** The system leverages real-time data, providing farmers with up-to-date recommendations based on current environmental and soil conditions.

DISADVANTAGES:

1. **Data Dependency:** The accuracy of the system heavily relies on the availability and quality of data such as soil composition, weather, and crop type, which may not always be accessible or reliable in some regions.
2. **Initial Setup Costs:** Implementing the system may require an initial investment in data collection tools, software development, and farmer training.
3. **Limited to Available Data:** The system can only provide recommendations based on the data it has, meaning if some variables (e.g., unrecorded soil conditions) are not considered, recommendations might not be optimal.
4. **Technology Barriers:** Farmers in rural or low-tech areas may face challenges in accessing or using the system due to limited internet connectivity or lack of technological knowledge.
5. **Model Limitations:** While AI models are powerful, they are not foolproof and may not always account for all the unique, localized factors that affect fertilizer needs, such as pests, diseases, or specific weather anomalies.

8.CONCLUSION

- The Fertilizer Recommendation System for Agriculture using AI successfully integrates advanced machine learning algorithms and data-driven approaches to optimize fertilizer usage in farming. By analyzing key parameters such as soil type, crop type, weather conditions, and nutrient levels, the system provides precise fertilizer recommendations tailored to specific agricultural needs. This not only enhances crop yield but also promotes sustainable agricultural practices by minimizing overuse or misuse of fertilizers.
- The project demonstrates the potential of AI in addressing critical challenges in agriculture, such as reducing environmental impact and increasing food security. The system's scalability ensures it can be adapted for various regions, crops, and farming practices. Additionally, its user-friendly interface ensures accessibility for farmers, enabling them to make informed decisions with minimal technical expertise.
- Future work can focus on improving the system by incorporating real-time IoT data, expanding the database with more regional-specific information, and integrating pest and disease management for a holistic agricultural decision-support system. With continued refinement, this AI-based solution can play a pivotal role in revolutionizing modern farming practices.

9.FUTURE SCOPE

- 1. Integration with IoT Devices:** The system can be enhanced by incorporating data from IoT devices such as soil sensors, weather stations, and drones. This will allow real-time monitoring of soil health, moisture levels, and crop growth, leading to more accurate fertilizer recommendations.
- 2. Incorporation of Regional and Crop-Specific Data:** Expanding the database with regional soil profiles, crop varieties, and local climatic conditions can improve the accuracy and relevance of recommendations for farmers in diverse geographical locations.
- 3. Mobile Application Development:** Developing a mobile application will make the system more accessible to farmers, even in remote areas. The app could support multiple languages, provide voice assistance, and allow offline usage for convenience.
- 4. Pest and Disease Management:** Integrating pest and disease prediction models into the system can offer comprehensive solutions by advising on fertilizers and other protective measures to safeguard crops.
- 5. Machine Learning Model Enhancement:** Advanced AI techniques, such as deep learning, could be utilized to further improve the prediction accuracy by analyzing complex patterns in the data. Additionally, transfer learning could be explored to adapt the system to new regions or crops with minimal retraining.
- 6. Sustainability Metrics:** Incorporating sustainability metrics to evaluate and suggest eco-friendly fertilizers and practices can help reduce environmental impact, contributing to sustainable agriculture.
- 7. Integration with Market Insights:** By integrating market trends and fertilizer price data, the system can recommend cost-effective fertilizer options, helping farmers optimize their expenses while maintaining productivity.
- 8. Policy and Government Support:** Collaboration with government bodies can facilitate subsidies or incentives for farmers using the system. It can also help in aligning the recommendations with regional agricultural policies.
- 9. Community Feedback and Learning:** Building a feedback loop where farmers can share their experiences and outcomes will allow the system to learn and improve over time, ensuring practical applicability and trust among users.
- 10. Global Expansion:** Scaling the system for global application by accommodating diverse agricultural practices, crops, and languages can make it a universal tool for precision farming worldwide.

10.APPENDIX

10.1 SOURCE CODE:

Index.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Plant Disease Prediction</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-color: #f9f9f9;
    }
    header {
      background-color: #333;
      color: white;
      padding: 15px;
      font-size: 24px;
      text-align: center;
      display: flex;
      justify-content: space-between;
      align-items: center;
    }
    header a {
      color: white;
      text-decoration: none;
      padding: 10px 20px;
      background-color: #444;
      border-radius: 5px;
    }
    header a:hover {
      background-color: #555;
    }
    .content {
      padding: 20px;
      text-align: center;
      background-color: #d4d7a3;
      color: #333;
    }
    .content img {
      width: 300px;
```

```

    height: auto;
    border-radius: 50%;
    float: right;
    margin: 20px;
}
.content h1 {
    font-size: 36px;
    margin-top: 20px;
}
.content p {
    font-size: 18px;
    line-height: 1.6;
    margin-top: 10px;
}
.btn {
    display: inline-block;
    padding: 10px 20px;
    margin-top: 20px;
    color: white;
    background-color: #5cb85c;
    text-decoration: none;
    border-radius: 5px;
}
.btn:hover {
    background-color: #4cae4c;
}
</style>
</head>
<body>
    <header>
        <div>Plant Disease Prediction</div>
        <div>
            <a href="/">Home</a>
            <a href="/predict">Predict</a>
        </div>
    </header>
    <div class="content">
        <h1>Detect if your plant is infected!!</h1>
        <p>
            Agriculture is one of the major sectors worldwide. Over the years, it has developed, and the use
            of new technologies and equipment replaced almost all traditional farming methods.
            This application helps farmers detect diseases by observing spots on leaves, which saves effort
            and labor costs.
        </p>
        
        <a href="/predict" class="btn">Predict Now</a>
    </div>
</body>
</html>

```

Predict.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Plant Disease Prediction - Upload Image</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f9f9f9;
    }
    header {
      background-color: #333;
      color: white;
      padding: 15px;
      text-align: center;
    }
    .content {
      padding: 20px;
      text-align: center;
      background-color: #d4d7a3;
    }
    .btn {
      display: inline-block;
      padding: 10px 20px;
      color: white;
      background-color: #5cb85c;
      text-decoration: none;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <header>
    <h1>Upload Plant Image</h1>
  </header>
  <div class="content">
    <h2>Select an image of your plant to predict the disease</h2>
    <form action="/predict" method="post" enctype="multipart/form-data">
      <input type="file" name="image" accept="image/*" required>
      <br><br>
      <button type="submit" class="btn">Submit Image</button>
    </form>
  </div>
</body>
</html>
```

Result.HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Prediction Result</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      background-color: #f9f9f9;
    }
    header {
      background-color: #333;
      color: white;
      padding: 15px;
      text-align: center;
    }
    .content {
      padding: 20px;
      text-align: center;
      background-color: #d4d7a3;
    }
    .btn {
      display: inline-block;
      padding: 10px 20px;
      color: white;
      background-color: #5cb85c;
      text-decoration: none;
      border-radius: 5px;
    }
    .btn:hover {
      background-color: #4cae4c;
    }
  </style>
</head>
<body>
  <header>
    <h1>Prediction Result</h1>
  </header>
  <div class="content">
    <h2>Precaution for the Disease:</h2>
    <p>{{ precaution }}</p>
    <br><br>
    <a href="/" class="btn">Back to Home</a>
    <a href="/predict" class="btn">Predict Another</a>
  </div>
</body>
</html>
```

App.py

```
import os
import numpy as np
import pandas as pd
from flask import Flask, render_template, request
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
from werkzeug.utils import secure_filename
from tensorflow.keras.utils import load_img, img_to_array

# Initialize Flask app
app = Flask(__name__)

# Load your trained model (adjust the model path if necessary)
model = load_model("vegetable.h5")

# Load precautions Excel file
df_precautions = pd.read_excel('precautions veg.xlsx') # Adjust the file path if necessary

# Ensure the upload folder exists
UPLOAD_FOLDER = 'uploads'
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)

# Define allowed image extensions
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}

# Function to check if the file extension is valid
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# Home route
@app.route('/')
def home():
    return render_template('index.html')

# Prediction route (GET and POST methods)
@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == 'POST':
        # Check if a file is part of the request
        if 'image' not in request.files:
            return "No file part"

        file = request.files['image']

        # If the user does not select a file, browser also submits an empty part without filename
        if file.filename == "":
            return "No selected file"

        # If a valid file is selected, process it
        if file and allowed_file(file.filename):
            # Save the file securely
            filename = secure_filename(file.filename)
            file_path = os.path.join(UPLOAD_FOLDER, filename)
            file.save(file_path)

            # Load and preprocess the image
            img = load_img(file_path, target_size=(64, 64), color_mode='grayscale') # Convert to grayscale
            x = img_to_array(img)
            x = np.expand_dims(x, axis=-1) # Ensure channel dimension
```

```

x = np.expand_dims(x, axis=0) # Add batch dimension

# Make prediction
pred = np.argmax(model.predict(x), axis=-1) # Get the predicted class index

# Retrieve corresponding precaution from the Excel file
predicted_class_id = pred[0]
precaution = df_precautions.iloc[predicted_class_id]['Precautions']

return render_template('result.html', precaution=precaution)
else:
    return "File type not allowed"
else:
    return render_template('predict.html')

# Run the app
if __name__ == "__main__":
    app.run(debug=True)

```

CODE SNIPPETS

DATA COLLECTION

Importing necessary libraries

```
In [40]: from keras.models import Sequential
from keras.layers import Dense
from keras.layers import Convolution2D
from keras.layers import MaxPooling2D
from keras.layers import Flatten
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import load_model
import pandas as pd
from tensorflow.keras.utils import load_img, img_to_array
import numpy as np
from keras.preprocessing import image
from tensorflow.keras.preprocessing.image import img_to_array
from tensorflow.keras.models import load_model
import numpy as np
```

Preprocess the Images

```
train_datagen = ImageDataGenerator(
    rescale=1./255,
    shear_range=0.2,
    zoom_range=0.2,
    horizontal_flip=True)

test_datagen = ImageDataGenerator(rescale=1./255)
```

DATA PREPROCESSING

```
x_train = train_datagen.flow_from_directory(
    'veg-dataset/veg-dataset/train_set',
    target_size=(128, 128),
    batch_size=16,
    class_mode='categorical'
)
```

Found 11386 images belonging to 9 classes.

```
x_test = test_datagen.flow_from_directory('veg-dataset/veg-dataset/test_set',
    target_size = (128,128),
    batch_size = 16,
    class_mode = 'categorical')
```

Found 3837 images belonging to 9 classes.

Initialize the model

```
model = Sequential()
```

Add the convolution layer

```
model.add(Convolution2D(32, (3, 3), input_shape=(128, 128, 3), activation='relu'))
```

C:\Users\ganin\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:107: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Add the pooling layer

```
model.add(MaxPooling2D(pool_size=(2,2)))
```

Add the flatten layer

```
model.add(Flatten())
```

Adding the dense layers

```
model.add(Dense(units=300, kernel_initializer='uniform', activation='relu'))
model.add(Dense(units=150, kernel_initializer='uniform', activation='relu'))
model.add(Dense(units=75, kernel_initializer='uniform', activation='relu'))
model.add(Dense(units=9, kernel_initializer='uniform', activation='softmax'))
```

Compile the model

```
: model.compile(loss='categorical_crossentropy',optimizer='adam',metrics=['accuracy'])
```

```
: model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_1 (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d_1 (MaxPooling2D)	(None, 63, 63, 32)	0
flatten_1 (Flatten)	(None, 127008)	0
dense_4 (Dense)	(None, 300)	38,102,700
dense_5 (Dense)	(None, 150)	45,150
dense_6 (Dense)	(None, 75)	11,325
dense_7 (Dense)	(None, 9)	684

Total params: 38,160,755 (145.57 MB)

Trainable params: 38,160,755 (145.57 MB)

Non-trainable params: 0 (0.00 B)

```
: num_classes = model.output_shape[-1] # This gives the number of output units (predicted classes)
print(f"Number of predicted classes: {num_classes}")
```

Number of predicted classes: 9

Fit and save the model

```
model.fit(x_train, steps_per_epoch=89, epochs=20, validation_data=x_test, validation_steps=27)
```

```
Epoch 1/20
89/89 ————— 72s 807ms/step - accuracy: 0.8112 - loss: 0.5094 - val_accuracy: 0.6389 - val_loss: 1.2222
Epoch 2/20
89/89 ————— 69s 781ms/step - accuracy: 0.8152 - loss: 0.5468 - val_accuracy: 0.6921 - val_loss: 1.1054
Epoch 3/20
89/89 ————— 69s 781ms/step - accuracy: 0.8188 - loss: 0.4976 - val_accuracy: 0.6736 - val_loss: 1.3312
Epoch 4/20
89/89 ————— 68s 765ms/step - accuracy: 0.8492 - loss: 0.4161 - val_accuracy: 0.6852 - val_loss: 1.1347
Epoch 5/20
89/89 ————— 90s 1s/step - accuracy: 0.8351 - loss: 0.4932 - val_accuracy: 0.7870 - val_loss: 0.8385
Epoch 6/20
89/89 ————— 70s 788ms/step - accuracy: 0.8332 - loss: 0.4844 - val_accuracy: 0.7500 - val_loss: 0.8168
Epoch 7/20
89/89 ————— 66s 743ms/step - accuracy: 0.8348 - loss: 0.4528 - val_accuracy: 0.7546 - val_loss: 0.8438
Epoch 8/20
89/89 ————— 65s 733ms/step - accuracy: 0.8618 - loss: 0.4457 - val_accuracy: 0.7222 - val_loss: 0.8798
Epoch 9/20
89/89 ————— 3s 34ms/step - accuracy: 0.0000e+00 - loss: 0.0000e+00 - val_accuracy: 0.7008 - val_loss: 0.9490
Epoch 10/20
89/89 ————— 69s 772ms/step - accuracy: 0.8383 - loss: 0.4441 - val_accuracy: 0.6968 - val_loss: 1.1443
Epoch 11/20
89/89 ————— 72s 815ms/step - accuracy: 0.8429 - loss: 0.4367 - val_accuracy: 0.7431 - val_loss: 0.9218
Epoch 12/20
89/89 ————— 77s 864ms/step - accuracy: 0.8524 - loss: 0.4097 - val_accuracy: 0.8333 - val_loss: 0.6916
Epoch 13/20
89/89 ————— 67s 750ms/step - accuracy: 0.8379 - loss: 0.4115 - val_accuracy: 0.7917 - val_loss: 0.8704
Epoch 14/20
89/89 ————— 65s 731ms/step - accuracy: 0.8833 - loss: 0.3533 - val_accuracy: 0.7616 - val_loss: 1.0049
Epoch 15/20
89/89 ————— 65s 726ms/step - accuracy: 0.8372 - loss: 0.4864 - val_accuracy: 0.8148 - val_loss: 0.8318
Epoch 16/20
89/89 ————— 65s 729ms/step - accuracy: 0.8671 - loss: 0.3785 - val_accuracy: 0.7106 - val_loss: 1.2380
Epoch 17/20
89/89 ————— 64s 718ms/step - accuracy: 0.8342 - loss: 0.4357 - val_accuracy: 0.7176 - val_loss: 1.3334
Epoch 18/20
89/89 ————— 3s 29ms/step - accuracy: 0.0000e+00 - loss: 0.0000e+00 - val_accuracy: 0.7323 - val_loss: 1.1846
Epoch 19/20
89/89 ————— 67s 743ms/step - accuracy: 0.8891 - loss: 0.3074 - val_accuracy: 0.6968 - val_loss: 1.1625
Epoch 20/20
5/89 ————— 53s 631ms/step - accuracy: 0.9248 - loss: 0.3165
```

```
model.save('vegetable.h5')
```

load the saved model

```
model = load_model('vegetable.h5')
print(model.input_shape)
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

(None, 128, 128, 3)

```
from tensorflow.keras.preprocessing.image import load_img, img_to_array
import numpy as np

# Load and preprocess the input image
file_path = r'C:\Users\ganin\OneDrive\Desktop\major\Veg-dataset\Veg-dataset\test_set\Tomato___Septoria_leaf_spot\c949b

# Load the image in RGB mode and resize it
img = load_img(file_path, target_size=(128, 128), color_mode='rgb')

# Convert the image to a numpy array
x = img_to_array(img) # Shape: (64, 64, 3)
x = x / 255.0 # Normalize the pixel values to [0, 1]
x = np.expand_dims(x, axis=0) # Add batch dimension, final shape: (1, 64, 64, 3)

# Check the shape
print("Input shape for prediction:", x.shape)
```

Input shape for prediction: (1, 128, 128, 3)

Test The Model

```
: model = load_model("vegetable.h5")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

```
: x = image.img_to_array(img)

x = np.expand_dims(x,axis = 0)
```

```
: pred
```

```
: array([[0.10883133, 0.11523615, 0.10956724, 0.11120898, 0.11280238,
          0.10979958, 0.10691235, 0.11261076, 0.11303122]], dtype=float32)
```

10.2 Github & project Demo Link:

Github link: [itsganeshhere/Fertilizer-Recommendation](https://github.com/itsganeshhere/Fertilizer-Recommendation)